AntiGravity - Hyperlocal Service Booking MVP

Business Requirement Document (BRD)

Project Code: AG-WEB-MVP **Target City:** Ahmedabad **Timeline:** 14-day realistic build **Business Model:** Commission-based (Default 15%) **Platform:** Web Application (Responsive Mobile & Desktop)

---

1. Objective & Scope

The objective is to pivot from a mobile-first approach to a web-based system for the Anti Gravity hyperlocal service platform. The MVP focuses strictly on core booking mechanics without introducing complex wallets or external payment gateways to meet the aggressive 14-day timeline.

The system will facilitate connecting Customers in Ahmedabad with local verified Service Providers (Electricians, Cleaners, etc.) while providing platform owners with a robust command center to manage operations and track 15% booking commissions.

PHASE 1 – WEB SYSTEM ARCHITECTURE

1.1 Core Web Modules

1. **Customer Web App:** A responsive web application optimized for mobile browsers allowing users to discover services and book providers.
2. **Provider Web Panel:** A secure portal for service partners to accept/reject jobs, view booking details, and track payouts.
3. **Admin Dashboard:** The primary operational control system for the business owners to govern the entire marketplace.

1.2 Layered Architecture

- **Frontend Layer:** React.js / Vite / Tailwind CSS.
- **API / Service Layer:** Firebase Client SDKs for direct frontend-to-backend communication. Firebase Cloud Functions for secure server-side logic (commissions, aggregations).
- **Database Layer:** Firebase Cloud Firestore (NoSQL).
- **Auth Layer:** Firebase Authentication (Phone/OTP based).
- **Notification Layer:** Firebase Cloud Messaging (FCM) for web push notifications.

1.3 Role-Based Access Logic

- **Customer Scope:** Read/Write personal profile, Read public categories/providers, Read/Write personal bookings.
- **Provider Scope:** Read/Write personal profile & status, Read assigned bookings, Update specific status flags on active bookings, Read personal earnings.
- **Admin Scope:** Full global Read/Write access across all collections (Users, Providers, Bookings, Categories, Commissions).

PHASE 2 – ADMIN DASHBOARD STRUCTURE (Detailed)

A. Dashboard Overview Page

Purpose: High-level operational health snapshot. **Metrics Displayed:**

- Total Bookings, Pending Bookings, Completed Bookings, Cancellation Rate, Total Revenue, Commission Earned, Active Providers. **Calculation Logic:** Computed via Firestore `count()` queries on the `bookings` collection. Financials (Total Revenue, Commission Earned) are served from a single, daily-updated aggregated document (`platform_stats`) maintained via Cloud Functions to minimize document reads.

B. Provider Management Module

Features:

- Data table listing all registered providers.
- Approve / Reject action buttons for pending registrations.
- Suspend / Reactivate action buttons for active/suspended providers.
- View provider profile and historical bookings. **Firestore Updates:** Updates the `status` ('active', 'suspended', 'pending', 'rejected') field in the `/providers` collection. **Edge Case Handling:** Suspending an active provider triggers a backend check to flag any currently 'pending' or 'accepted' bookings for Admin manual reassignment.

C. Booking Monitoring Module

Features:

- Live list of all bookings across the platform, reverse sorted by most recent.
- Filters: Date range, Status, Category, Provider.
- Financial Display: Show final negotiated amount or original request amount.
- Manual Status Override (Admin god-mode to force cancel or complete a stuck booking).
- View booking timeline history. **Query Logic:** `db.collection('bookings').where('status', '==', filter).orderBy('createdAt', 'desc')` . Requires composite indexes established in Firestore. **Validation:** Any manual override appends a log to the booking's `statusHistory` array detailing the Admin UID and timestamp.

D. Commission Dashboard

Display:

- List view of recent platform commissions generated per booking.
- Aggregated total commission for a selected date range.
- Aggregated commission generated per provider.
- CSV Export functionality. **Query Aggregation Logic:** Reads from a dedicated `/commissions` collection populated exclusively by a secure backend Cloud Function that runs when a booking status changes to `completed` .

E. User Management Module

Features:

- Data table of all registered consumers.
- View individual user booking history.
- Block user functionality (prevents further logins/bookings).
- Audit activity metrics (joined date, lifetime total bookings).

PHASE 3 – CUSTOMER WEB APP STRUCTURE

Accessible via `/customer`

Screens & Flow:

1. **Landing Page:** Hero banner, Trust signals, Popular Categories grid. Directs user to login or browse.
2. **Login (OTP):** Firebase Phone Auth UI (Mock OTP for MVP). Captures mobile number and verifies OTP.
3. **Home Page (`/customer/app`):** Personalized greeting, category selection, top-rated local providers.
4. **Category Listing:** Interactive cards to filter top providers by service.
5. **Provider Listing (Search Results):** Filtered view of active and approved providers matching a selected category.
6. **Provider Detail:** Provider initial avatar, ratings, total jobs completed, price estimations.
7. **Booking Form:** Captures exact service address and optional issue description via modal overlay.
8. **Booking Confirmation:** Success card and status entry placed into "Current Activity".
9. **Booking Status / History:** Live timeline (`pending` > `negotiating` > `accepted` > `completed`). Support for Accepting/Declining custom price proposals.
10. **Rating:** Interactive 5-star rating component appears in the "Past Bookings" list once the job is marked `completed` .

PHASE 4 – PROVIDER WEB PANEL STRUCTURE

Accessible via `/provider`

Screens & Flow:

1. **Login:** Firebase Phone Auth UI (Mock OTP for MVP).
2. **Dashboard (`/provider`):** Incoming leads and active jobs. Allows accepting, rejecting, or proposing custom prices with instant UI feedback.
3. **Active Booking Detail:** Shows customer address, price, and status. Includes a button to mark the job as "Completed".
4. **Earnings Dashboard (`/provider/earnings`):** Financial ledger showing clear breakdown of Gross Job Total, 15% Platform Fee, and Net Earnings.
5. **Profile (`/provider/profile`):** Read-only view of current ratings, jobs completed, registered phone, and active status.

Approval Gating Logic: If a provider is `pending` or `suspended` in the database, they are shown a persistent banner on their dashboard warning them of restricted access until Admin intervention.

PHASE 5 – ROLE-BASED ROUTING & AUTH

Implementation Logic:

- **Role Detection:** Utilizing `AuthContext` with database mapping for MVP development. Local phone mock auth associates local users with their roles (Admin, Provider, Customer).
- **Session Persistence:** Managed by mock mapping to `localStorage` (`mockDb.js`) syncing for state retention across roles.
- **Route Protection (React Router):**
  - `/admin/*` routes wrapped in `<AdminRoute>` .
  - `/provider/*` routes wrapped in `<ProviderRoute>` .
  - `/customer/*` routes wrapped in `<CustomerRoute>` , with generic root `/` redirecting to the `/customer` landing page.
- **Unauthorized Handling:** Directs intruders to the respective context `/login` page (Admin Login, Provider Login, or Customer Login).

PHASE 6 – BOOKING LOGIC (Web Context)

Lifecycle States:

1. **Pending:** Created by customer. Awaiting provider response.
    - *Path A:* Provider accepts -> shifts to `accepted`.
    - *Path B:* Provider proposes a new price -> shifts to `negotiating`.
    - *Path C:* Provider rejects -> triggers reassignment logic or cancels.
2. **Negotiating:** Provider proposed a new price.
    - *Path A:* Customer accepts new price -> shifts to `accepted`.
    - *Path B:* Customer declines new price -> shifts to `rejected`.
3. **Accepted:** Provider has committed.
    - *Path A:* Work finishes -> Provider clicks 'Complete' -> shifts to `completed`.
    - *Path B:* Customer cancels -> shifts to `cancelled`.
4. **Completed:** Terminal state. Triggers commission calculation.
5. **Cancelled / Rejected:** Terminal state. No commission.

Firestore Update Flow & Commission: When a Provider marks a job `completed` via the UI:

1. Client updates booking `status: 'completed'`.
2. A Firestore Cloud Function (`onUpdate`) detects this specific status change.
3. The server calculates 15% of the `bookingValue`.
4. The server creates a secure document in `/commissions`.
5. FCM pushes a "Job Finished & Receipt" notification to the Customer.

PHASE 7 – FIRESTORE SECURITY RULES

Core Philosophies (Web-Safe):

1. **Admin:** Full read/write over the entire database via `request.auth.token.admin == true` custom claim.
2. **Providers:** Cannot modify the `price`, `commission`, or `customerId` fields on a booking. They can only modify the `status` field of a booking where `resource.data.providerId == request.auth.uid`.
3. **Customers:** Can only read/write documents where `resource.data.customerId == request.auth.uid`.
4. **Commission Collection:** Strictly Read/Write locked to Admins and backend Cloud Functions only.

PHASE 8 – ADMIN UX DESIGN STRUCTURE

Design Approach: Clean, data-dense, minimalist UI optimized for desktop monitors. Utilizing standard table patterns with pagination and persistent sidebars.

Layout Architecture:

- **Sidebar Navigation (Left, Fixed):**
    - Dashboard Overview
    - Provider Fleet
    - Booking Monitor
    - Commission Reports
    - Consumer Management
    - Settings
- **Top Bar (Top, Fixed):**

- Global Search Input (Lookup by ID or Phone).
- Admin Profile / Avatar.
- Notification Bell (Alerts for reported providers or system errors).
- Secure Logout.

PHASE 9 – MVP DEPLOYMENT PLAN

Deployment Checklist:

1. [ ] **Firebase Initialization:** Provision Auth, Firestore, Storage, Hosting.
2. [ ] **Admin Initialization:** Manually script/inject Custom Claims (`admin: true`) to the founding team's auth UIDs.
3. [ ] **Environment Specs:** Tie `.env.production` pointing exclusively to the Prod Firebase project.
4. [ ] **Index Deployment:** Deploy `firestore.indexes.json` to support multi-field sorting on the Bookings dashboard.
5. [ ] **Security Rules:** Deploy the web-safe `firestore.rules` .
6. [ ] **Hosting Build:** Run `npm run build` . Configure `firebase.json` to serve `index.html` for all routes (SPA setup).
7. [ ] **Domain Mapping:** Connect custom domains (e.g., `admin.domain.com` routing to specific logic, or keeping it monolithic `domain.com/admin`).
8. [ ] **Go Live:** Fire `firebase deploy --only hosting` .